

N Queens

N-Queens Optimization Recording
https://drive.google.com/file/d/1Ppn1s_KM9TlMdXUj956NonDlECxkM6/view?usp=sharing

N Queens

Given a $N \times N$ chessboard, and N queens, design a recursive algorithm to place the queens on the chessboard such that no two queens attack each other – we say that two queens will attack each other when they are placed along the same row, column or diagonal.

Example

$N = 4$

	0	1	2	3
0		Q_0		
1				Q_1
2	Q_2			
3			Q_3	

$n=4$

	Q_0	Q_1	Q_2	Q_3
0				
1				
2				
3				

$r=0$

	Q_0	Q_1	Q_2	Q_3
0				
1				
2				
3				

4x4

$f(r)$

	0	j	N-1
0			
1			
2			
$\vdots$			
$r-1$			
r			
$r+1$			
$\vdots$			
$N-1$			

$N \times N$

if ($r == n$) {
 // you've built a
 valid config
 return;
}

init $r = 0$

you can place Q_r in either
 col 0, 1, 2, 3...n-1

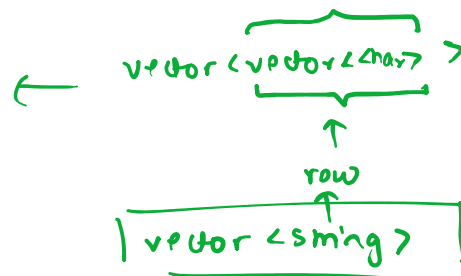
$0 \leq j \leq n-1$

$f(r+1)$

it is a fn that takes decision for Q_r to Q_{n-1}

$\Rightarrow$ to solve this all you've to do is decide for Q_r

decide the col in row r where you'll place Q_r



vector < vector < string > >

$n = 4$

" "

string s(n, '.');

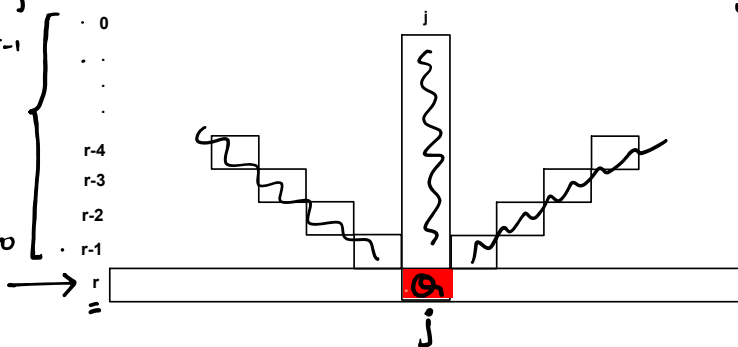
vector < string > board;

for (i = 1; i <= n; i++) {
 string s(n, '.');
 board.pb(s);
}

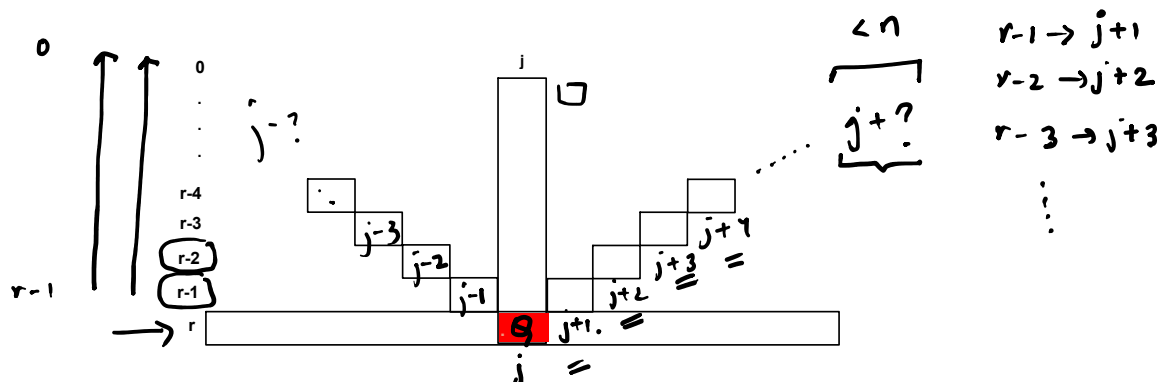
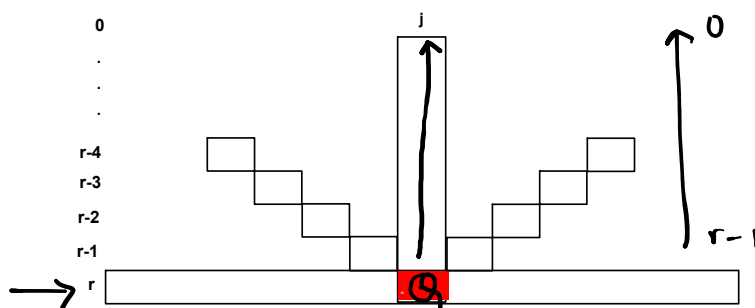
vector < vector < string > > allBoards;

$Q_i \quad 0 \leq i \leq r-1$

Q_0 to Q_{r-1}
are
placed
to some
coln of
row 0 to
 $r-1$



j th coln is valid
for Q_r only
when there is
no prev. placed
queen Q_i
 $0 \leq i \leq r-1$ along
the same coln,
left & right diag.
of the j th coln



[R][HW] Solve Sudoku

Solve Sudoku

Given a **partially filled 9x9 Sudoku Grid**, design a recursive algorithm to assign a **digit** in the range **1** to **9** to all the empty cells such that each row, column & $\sqrt{9} \times \sqrt{9}$ sub-grid contain **exactly one** instance of a digit.

Example

3	.	6	5	.	8	4	.	.
5	2	.	.	.	.	.	.	.
.	8	7	.	.	.	.	3	1
.	.	3	.	1	.	.	8	.
9	.	.	8	6	3	.	.	5
.	5	.	.	9	.	6	.	.
1	3	.	.	.	.	2	5	.
.	.	.	.	.	.	.	7	4
.	.	5	2	.	6	3	.	.

	0		j		N-1
0					
i					

i				
N-1				

N x N

3	.	6	5	.	8	4	.	.
5	2	.	.	.	.	.	.	.
.	8	7	.	.	.	.	3	1
.	.	3	.	1	.	.	8	.
9	.	.	8	6	3	.	.	5
.	5	.	.	9	.	6	.	.
1	3	.	.	.	.	2	5	.
.	.	.	.	.	.	.	7	4
.	.	5	2	.	6	3	.	.

Time $\#$ rec-calls $\times$ time spent in each call

Time & Space Complexity Analysis of Recursive Algorithms

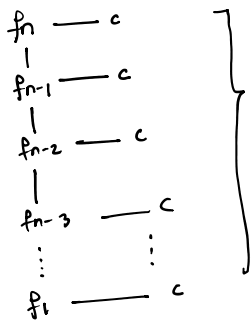
Back Substitution Method

```
void f(n) {
    // base case
    if(n == 1) {
        return;
    }
    // recursive case
    f(n-1);
}
```

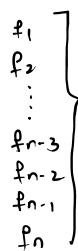
$$\begin{aligned}
 t(n) &= t(n-1) + c & \text{--- (1)} \\
 t(n-1) &= t(n-2) + c & \text{--- (2)} \\
 t(n-2) &= t(n-3) + c & \text{--- (3)} \\
 &\vdots \\
 t(1) &= c
 \end{aligned}$$

$$\begin{aligned}
 t(n) &= t(n-1) + c \\
 &= (t(n-2) + c) + c \\
 &= t(n-2) + 2c \\
 &= t(n-3) + 3c \\
 &\vdots \\
 &= t(n-k) + kc \\
 &= t(1) + (n-1)c \\
 &= c + (n-1)c \\
 &= nc \sim O(n)
 \end{aligned}$$

$n-k=1$
 $\therefore k=n-1$



$$n \cdot c \sim O(n)$$



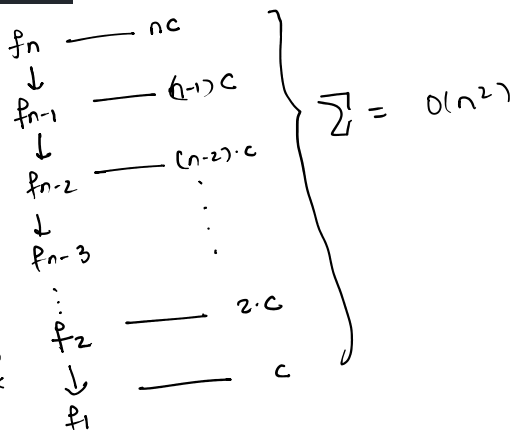
$$\text{space: } O(n)$$

```
void f(n) {
    // base case
    if(n == 1) {
        return;
    }
    // recursive case
    for(int i=0; i<n; i++){
        f(n-1);
    }
}
```

$$\begin{aligned}
 t(n) &= t(n-1) + nc & \text{--- (1)} \\
 t(n-1) &= t(n-2) + (n-1)c & \text{--- (2)} \\
 t(n-2) &= t(n-3) + (n-2)c & \text{--- (3)} \\
 &\vdots \\
 t(1) &= c
 \end{aligned}$$

$$\begin{aligned}
 t(n) &= t(n-1) + nc \\
 &= t(n-2) + (n-1)c + nc \\
 &= t(n-3) + (n-2)c + (n-1)c + nc \\
 &\vdots \\
 &= t(n-k) + (n-(k-1))c + \dots + (n-2)c + (n-1)c + nc \\
 &= t(1) + c + 2c + \dots + (n-2)c + (n-1)c + nc \\
 &= c [1 + 2 + \dots + n] \\
 &= c \left[\frac{n(n+1)}{2} \right] \sim n^2 \\
 &= n^2 c \sim O(n^2)
 \end{aligned}$$

$n-k=1$
 $k=n-1$



```
int f(n) {
    // base case
    if(n == 1) {
        return 1;
    }
    // recursive case
    f(n/2);
}
```

$$\begin{aligned}
 t(n) &= 2t(n/2) + c & \text{--- (1)} \\
 t(n/2) &= 2t(n/4) + c & \text{--- (2)} \\
 t(n/4) &= 2t(n/8) + c & \text{--- (3)} \\
 &\vdots
 \end{aligned}$$

$$\begin{aligned}
 t(n) &= 2t(n/2) + c \\
 &= 2(2t(n/4) + c) + c \\
 &= 2^2 t(n/4) + 2c + c \\
 &= 2^2 (2t(n/8) + c) + 2c + c
 \end{aligned}$$


```

if(n == 1) {
    return 1;
}

// recursive case
return f(n/2) + f(n/2);

```

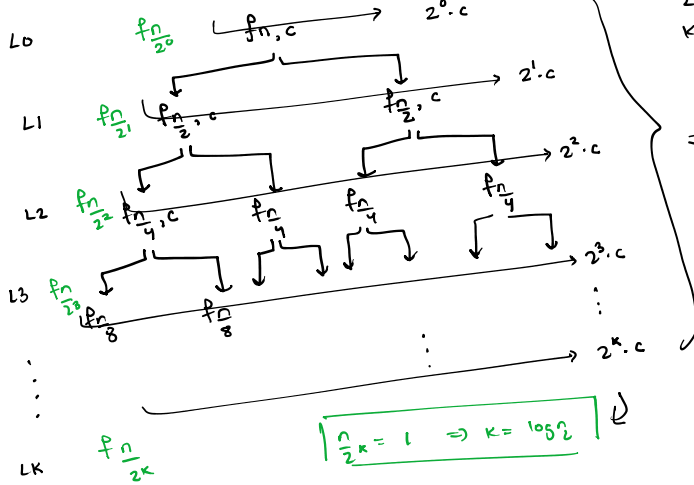
$$\begin{aligned}
 t(n/4) &= 2t(n/8) + c \\
 &\vdots \\
 t(1) &= c
 \end{aligned}$$

$$\begin{aligned}
 &= 2^k t\left(\frac{n}{2^k}\right) + 2^k c \\
 &= 2^k \left(2t\left(\frac{n}{8}\right) + c \right) + 2^k c \\
 &= 2^3 t\left(\frac{n}{2^3}\right) + 2^2 c + 2^1 c + 2^0 c \\
 &\vdots \\
 &= 2^k t\left(\frac{n}{2^k}\right) + \dots + 2^2 c + 2^1 c + 2^0 c
 \end{aligned}$$

$$\begin{aligned}
 \frac{n}{2^k} &= 1 \\
 2^k &= n \\
 k &= \log_2 n
 \end{aligned}$$

$$\begin{aligned}
 \Sigma &= (2^{k+1} - 1) \cdot c \\
 &\sim O(n)
 \end{aligned}$$

$$\begin{aligned}
 &= 2^k t(1) + \dots + 2^2 c + 2^1 c + 2^0 c \\
 &= 2^k \cdot c + \dots + 2^2 c + 2^1 c + 2^0 c \\
 \Rightarrow &= c [2^0 + 2^1 + \dots + 2^k] \\
 &= c [2^{k+1} - 1] \\
 &= c [2 \cdot 2^k - 1] \\
 &= c [2 \cdot 2^{\log_2 n} - 1] \\
 &= c [2n - 1]
 \end{aligned}$$



space: $O(\log n)$ due to fn call stack

from each level we can have ≤ 1 fn call on the call stack

$$\begin{aligned}
 &= cL \\
 &= 2^{\log_2 n} \cdot c \sim O(n)
 \end{aligned}$$

```

void f(n) {
    // base case
    if(n == 1) {
        return;
    }

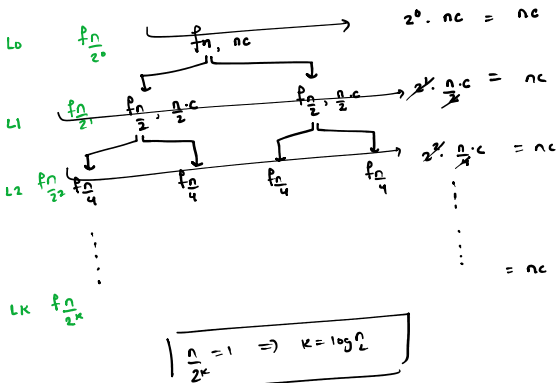
    // recursive case
    f(n/2);
    f(n/2);
    g(n); // assume it runs in O(n) time
}

```

$$\begin{aligned}
 t(n) &= 2t(n/2) + n \cdot c \quad (1) \\
 t(n/2) &= 2t(n/4) + n/2 \cdot c \quad (2) \\
 t(n/4) &= 2t(n/8) + n/4 \cdot c \quad (3) \\
 &\vdots \\
 t(1) &= c
 \end{aligned}$$

$$\begin{aligned}
 \frac{n}{2^k} &= 1 \\
 2^k &= n \\
 k &= \log_2 n
 \end{aligned}$$

$$\begin{aligned}
 &= 2^{\log_2 n} t(1) + \log_2 n \cdot n \cdot c \\
 &= n \cdot c + n \log_2 n \cdot c \\
 &= O(n \log n)
 \end{aligned}$$



space: $O(\log n)$ + $O(n)$

due to fn call stack

due to temp array in merge

$$\frac{n}{2^k} = 1 \Rightarrow k = \log_2 n$$

$$\begin{aligned}
 &(k+1) * nc \\
 &\Downarrow \\
 &\# \text{ levels} \quad \text{time spent on each level} \\
 &= knc + nc \\
 &= n \log_2 n + n \\
 &\sim O(n \log n)
 \end{aligned}$$

$$\begin{array}{r} \# -1) \\ \hline -1 \\ \hline 2^{k+1} - 1) \\ \hline 2^{-1} \end{array}$$


```
int f(n) {  
    // base case  
    if(n == 0 || n == 1) {  
        return n;  
    }  
    // recursive case  
    return f(n-1) + f(n-2);  
}
```

